



APSCO

LINKU Project

CAMERA SYSTEM WORKING DOCUMENT

"MASTER WORK PLAN: CAMERA SYSTEM AND TETHER RECONSTRUCTION"

September 20, 2026

Master Work Plan: Camera System and Tether Reconstruction

Prepared by:

¹ Pontificia Universidad Catolica del Peru (PUCP)

Author(s): Jose Sifuentes¹

Status: Living planning document, no fixed dates yet

Issue: 1

Revision: 0

Abstract

This document consolidates, into a single work plan, every pending objective currently spread across the project's technical deliverables (Blender Simulation Realism Validation, Rotation/Rolling-Shutter Experiment, Camera System Report) together with six tracking areas explicitly requested for mapping: new camera prototype, Blender simulation testing, power consumption, mode of use, data volume produced, and tether reconstruction.

No dates are assigned yet. Each chapter instead states the current, verified status of one area, its objective, and the ordered, actionable steps needed to reach that objective. A closing chapter maps dependencies between chapters, since several tasks share the same blocker (most notably, stereo camera recalibration).

Contents

1	Introduction	6
1.1	Purpose	6
1.2	How to read this document	6
1.3	What this document is not	6
2	Tether Reconstruction	7
2.1	Current status	7
2.2	Objective	7
2.3	Step-by-step tasks	7
3	New Camera Prototype	9
3.1	Current status	9
3.2	Objective	9
3.3	Step-by-step tasks	9
4	Blender Simulation Testing	11
4.1	Current status	11
4.2	Objective	11
4.3	Step-by-step tasks	11
5	Rotation and Rolling-Shutter Correction	13
5.1	Current status	13
5.2	Objective	13
5.3	Step-by-step tasks	13
6	Power Consumption	15
6.1	Current status	15
6.2	Objective	15
6.3	Step-by-step tasks	15
7	Mode of Use and Data Volume	16
7.1	Mode of use	16
7.1.1	Current status	16
7.1.2	Objective	16
7.1.3	Step-by-step tasks	16
7.2	Data volume produced	17
7.2.1	Current status	17
7.2.2	Objective	17
7.2.3	Step-by-step tasks	17
8	Stereo Camera Recalibration	18
8.1	Current status	18
8.2	Objective	18
8.3	Step-by-step tasks	18
9	Dependency Map and Master Checklist	19
9.1	Why order matters more than dates here	19

9.2	Dependency diagram	19
9.3	Master checklist	19
10	Summary	21

List of Figures

9.1	Dependency map. Green: can start immediately, no shared blocker. Orange: the one recurring physical blocker. Blue: waits on that blocker.	19
-----	---	----

List of Tables

9.1 Master checklist across all tracking areas. 20

Chapter 1

Introduction

1.1 Purpose

Two working documents already define detailed objectives and methodology for specific parts of the camera system: the Blender Simulation Realism Validation report and the Rotation/Rolling-Shutter Experiment report. Separately, six areas were identified as needing to be explicitly mapped and tracked: new camera prototype, Blender simulation testing, power consumption, mode of use, data volume produced, and tether reconstruction.

This document brings both sources together into one place. Rolling-shutter correction and Blender simulation validation each already have a dedicated report; rather than duplicating their content, this plan summarizes their current status and folds their remaining steps into the corresponding chapter here, so all seven areas can be read, tracked, and updated from a single document.

1.2 How to read this document

Each of Chapters 2 through 7 covers one tracking area with the same three-part structure:

- **Current status:** what already exists, verified directly against the code, data, and prior reports at the time of writing, not assumed from memory.
- **Objective:** the concrete end state that would close this area.
- **Step-by-step tasks:** an ordered list of actions needed to reach that objective. Steps are ordered by dependency, not by estimated duration, since no dates are assigned in this version of the document.

Chapter 8 covers stereo camera recalibration on its own, since it is not one of the six original tracking areas but blocks tasks inside several of them. Chapter 9 maps how the chapters depend on one another, so the plan can be worked in a sensible order even without fixed dates.

1.3 What this document is not

This is not a new technical report. Facts already established and cited with sources in the Blender Simulation Validation report, the Rotation/Rolling-Shutter report, and the Camera System Report are referenced here by name rather than re-derived or re-cited. Where a number in this document is new (for example, current dataset sizes), it was measured directly from the repository at the time of writing.

Chapter 2

Tether Reconstruction

2.1 Current status

The reconstruction pipeline is split into two AI components, treated as a single conceptual problem: given a segmented thread image, tangled or not, trace the correct path. This was an explicit correction made earlier in the project after the two were initially (and incorrectly) described as separate systems.

- **Model 1, cable segmentation:** implemented and trained. A Random Forest classifier using 9 pixel-level features (BGR, HSV, local contrast), trained on 6 stereo pairs expanded to 60 images through augmentation, reaches $F1 = 0.97$. This score is on the training set only; no held-out validation split has been run, so the real generalization performance is currently unknown. Trained weights exist at `data/models/mask_model.pkl`.
- **Model 2, unified wire-tracker and knot-resolver:** dataset-collection stage only, not yet trained. The proposed architecture reuses the existing classical search (depth-first search with backtracking, already implemented in `processing/smart_wire_tracker.py`) and replaces only its local branch-scoring rule with a learned one, rather than building a separate model from scratch.
- A classical, non-AI tracker is already functional today: `processing/smart_wire_tracker.py` performs DFS with backtracking and distance-transform-based adaptive width estimation. This is usable independently of Model 2 and is the fallback/baseline Model 2 must be benchmarked against.
- Dataset sizes, measured directly from the repository: `data/ai_dataset/mask_dataset/metadata.json` contains 6 labeled entries; `data/ai_dataset/path_dataset/metadata.json` contains 1 labeled entry. Out of 65 stereo capture sessions available under `data/captures/`, only these have been mined into training data. A path-dataset size of roughly 30 to 100 labeled examples has been estimated internally as the minimum needed to train even a small classifier for Model 2.

2.2 Objective

Reach a trained and validated Model 2 that resolves ambiguous crossings and knots at least as reliably as the classical DFS tracker handles simple, non-crossing cases, with Model 1's real accuracy confirmed on unseen images rather than only on its training set.

2.3 Step-by-step tasks

1. Mine the remaining, currently unused capture sessions (59 of 65) into `mask_dataset` and `path_dataset`. This is software-only, requires no hardware or recalibration, and is the single largest immediately actionable item in this chapter.

2. Split `mask_dataset` into a held-out validation set and recompute Model 1's F1 score on images it was not trained on, to replace the current training-only 0.97 figure with a real generalization estimate.
3. Prioritize growing `path_dataset` toward the 30 to 100 range using sessions that contain crossings or knot-like configurations specifically, not just simple single-strand cases, since those are the examples Model 2 needs to learn the branch-scoring rule from.
4. Implement Model 2 by keeping the existing DFS-with-backtracking structure in `smart_wire_tracker.py` and replacing only its local branch-scoring heuristic with a classifier trained on the grown `path_dataset`.
5. Benchmark Model 2 against the classical tracker baseline on the same held-out images, using mask coverage percentage, path length accuracy, and endpoint position accuracy as the comparison metrics, before adopting the learned version as the default.
6. Update the AI pipeline status table (currently in the Camera System Report) to reflect the new validated status of both models once the steps above are complete.

Chapter 3

New Camera Prototype

3.1 Current status

- The software stack currently assumes the Arducam HQ 477 (IMX477) sensor throughout: `config/camera_config.py` and `camera/stereo_camera.py` both reference it directly, and the libcamera capture command generation is built around its specific mode string.
- Three sensor options are already compared quantitatively in `calculate_cam/camera_dof_visualizer.py`: the current IMX477 baseline, the Arducam Hawk-eye (OV64A40, highest resolution), and the Arducam 12MP IMX708 Wide Angle B0310 (widest field of view, added to the comparator this project). Depth-of-field and field-of-view figures exist for all three, but no sensor choice has been recorded as final anywhere in the documentation.
- The official APSCO deliverable already references a 12MP IMX708-based module as the reference sensor for the mechanical mounting and sunshade CAD renders included in that report. Mechanical accommodation figures exist, but no physical camera prototype or housing build status is documented beyond those renders.
- No camera is currently physically installed in the dark enclosure (`SIMULATION/Simulation_DarkBox.blend` and the physical box are not synchronized with any live camera hardware at present).

3.2 Objective

Decide on the flight camera sensor using the tradeoffs already computed, then physically build, mount, and validate that camera in the real hardware end to end.

3.3 Step-by-step tasks

1. Review the DOF/FOV comparison already produced by `camera_dof_visualizer.py` for all three sensors at the tether's expected observation distance, and make an explicit, documented sensor decision (stay with IMX477, or move to Hawk-eye/IMX708).
2. Record the decision and its rationale in the Camera System Report, which is currently silent on this choice.
3. If a sensor other than IMX477 is selected: update `config/camera_config.py` and `camera/stereo_camera.py`, which are currently hardcoded around IMX477 assumptions, and confirm the generated libcamera command works for the new sensor.
4. Physically mount the chosen camera(s) in the dark enclosure, matching the mechanical CAD renders already produced for the APSCO report.
5. Recalibrate the stereo pair at full resolution with the finalized camera hardware (shared blocker, see [Chapter 8](#)).

6. Capture a validation stereo pair with the new hardware and confirm the reconstruction pipeline (Chapter 2) runs end to end on it without modification.

Chapter 4

Blender Simulation Testing

This chapter summarizes the Blender Simulation Realism Validation report. See that document for full methodology, literature review, and sourcing.

4.1 Current status

- Literature review and validation methodology: complete. The strategy follows the SISPO precedent (compare rendered output against real imagery using the project’s own reconstruction pipeline as the comparison instrument, not visual inspection alone).
- The Blender scene `SIMULATION/Simulation_DarkBox.blend` exists, with the enclosure’s matte-black material correctly applied to the real panel meshes and the Sun light’s irradiance and angular-diameter settings confirmed against Blender’s own documentation.
- One open bug remains unresolved: the tether curve renders dark and unlit regardless of light wattage (tested from 50W to 1000W). The self-shadowing/viewing-angle hypothesis has not yet been tested.
- A real reference stereo pair has already been selected for the laboratory-condition comparison: `data/captures/stereo_20260213_140535/left.jpg` and `right.jpg`.
- A physical dark-room solar illumination characterization plan exists (IEC 60904-9 target metrics, LED array recommendation, concrete component-level precedents), but the real light source in the box has not yet been measured.
- Matched-pair comparison and orbital-condition rendering are both pending, in that order, since the orbital render is only trusted once the laboratory-condition render is shown to agree with its real counterpart.

4.2 Objective

Establish, through the project’s own reconstruction pipeline, whether the Blender render is realistic enough to stand in for imagery that cannot be captured physically today (the tether under orbital illumination in vacuum).

4.3 Step-by-step tasks

1. Resolve the tether-rendering bug (dark tether regardless of wattage), either through continued headless script debugging or visual debugging once Blender’s GPU/driver access is confirmed working.
2. Characterize the real dark-enclosure light source: measure irradiance (W/m^2) at the tether location, spatial non-uniformity, and short-term temporal instability, using at minimum a lux meter.

3. Feed the measured real-light values into the Blender Sun/Point light setup for the laboratory-condition render, replacing the current placeholder value.
4. Render the laboratory-condition scene matching the already-selected reference photo pair's camera position, orientation, and tether pose.
5. Run the project's reconstruction pipeline (segmentation, endpoint detection, path tracing) on both the real photo and the Blender render, and compare the outputs directly.
6. If agreement is satisfactory, generate 3 to 5 additional matched pairs covering different tether poses (straight, curved, near self-crossing) before drawing conclusions from a single case.
7. Reconfigure the validated scene for orbital conditions (vacuum, AM0 illumination, no enclosure, unobstructed background) and render representative deployment geometries.

Chapter 5

Rotation and Rolling-Shutter Correction

This chapter summarizes the Rotation/Rolling-Shutter Experiment report. It is not one of the six originally listed tracking areas, but it is an active objective from an existing deliverable and is included here so the full plan lives in one place.

5.1 Current status

- A two-phase methodology is already defined: Phase 1 is a synthetic validation using existing static photographs with no hardware required; Phase 2 is a physical rotation rig.
- A rotation-tolerance formula has been derived: approximately $1.05^\circ/\text{s}$ using the software's current 15fps setting, or approximately $2.76^\circ/\text{s}$ using the IMX477 datasheet's stated 40fps 12-bit full-resolution capability.
- The exact row-scan-only readout time needed to convert the datasheet's frame-rate figure into a confirmed (not estimated) tolerance still requires the IMX477 Software Reference Manual, which is not publicly available, or a direct bench measurement.
- The correction method identified in the literature (Karpenko, Jacobs, Baek, Levoy) uses gyroscope data to correct rolling-shutter distortion; this has not yet been implemented.

5.2 Objective

Implement and validate a software-based rolling-shutter correction usable when the satellite is rotating during image capture.

5.3 Step-by-step tasks

1. Implement Phase 1: apply the gyroscope-based correction method to existing static photographs with synthetically injected rotation, to validate the correction algorithm itself without needing rotation hardware.
2. Attempt to obtain the IMX477 Software Reference Manual, or bench-measure the actual row readout time directly, to replace the two estimated tolerance figures with a confirmed one.
3. Build the Phase 2 physical rotation rig: mount a camera on a rotating stage with a known, controllable angular velocity.
4. Capture real images of a static scene while rotating, and apply the same correction algorithm validated in Phase 1.

5. Compare corrected against uncorrected images quantitatively, using the reconstruction pipeline's output agreement against a static (non-rotating) reference as the metric.

Chapter 6

Power Consumption

6.1 Current status

- Camera sensor power is quantified: the IMX477 datasheet gives approximately 269 mW (typical) to 382 mW (maximum) per camera at 30fps full resolution, so approximately 540 to 760 mW combined for the stereo pair, sensors only. This figure comes from a datasheet table watermarked confidential by the manufacturer; citing it was an explicit, informed decision made earlier in this project, documented in the Rotation/Rolling-Shutter report’s bibliography.
- The CM5 board’s own processing-side power draw is not characterized at all: no bench measurement or datasheet-derived figure exists for it in any project document.
- No complete per-stage power budget exists (idle, preview streaming, full-resolution capture, AI/processing). This is an open item already flagged as pending in the Camera System Report.
- No comparison against the satellite’s EPS-side power allocation for the camera subsystem has been made; the EPS/OBC integration code (outside this repository, in the `INTEGRACION_EPS_OBC` working directory) has not yet been reviewed for this purpose.

6.2 Objective

Produce a complete power budget covering every relevant operational stage and every subsystem that draws power during camera operation, not only the camera sensors.

6.3 Step-by-step tasks

1. Obtain or bench-measure the CM5’s own power draw at each operational stage: idle, preview streaming, full-resolution capture, and AI/processing (segmentation and tracking inference).
2. Combine the CM5 figures with the already-known camera sensor figures to produce a single per-stage total power table.
3. Cross-check the resulting figures against the EPS subsystem’s power allocation for the camera system, reviewing the `INTEGRACION_EPS_OBC` project for any existing power-budget documentation or constraints.
4. Document the final table in the Camera System Report, replacing the currently open power-budget todo item.

Chapter 7

Mode of Use and Data Volume

These two tracking areas are covered in one chapter because the achievable data volume directly constrains what capture cadence is realistic, and vice versa. Each keeps its own status, objective, and task list so both remain individually traceable.

7.1 Mode of use

7.1.1 Current status

- Only qualitative statements currently exist: the APSCO report states capture rate is higher during satellite separation and lower during stable observation, that saturated or non-informative images are discarded before downlink, and that a compression plus prioritized selective-downlink scheme is used rather than continuous raw downlink.
- No explicit trigger-condition table or command-sequence specification for camera capture exists yet.
- Integration with the OBC/EPSC command interface (in the separate `INTEGRACION_EPS_OBC` and `TRANSMITER_UHF_H7` projects) has not been reviewed for this plan and is not yet cross-referenced from the camera system documentation.

7.1.2 Objective

Define precisely when and how often the camera captures images, including the trigger mechanism, and connect that definition to the OBC command interface.

7.1.3 Step-by-step tasks

1. Define explicit capture cadence per mission phase (separation, stabilization, stable observation) as concrete numbers, replacing the current qualitative "higher/lower" description.
2. Define the trigger mechanism: autonomous (timer- or event-based) versus ground-commanded, and review the OBC integration code in `INTEGRACION_EPS_OBC` for the existing command interface this would need to use.
3. Define the onboard image-quality pre-filter logic: the concrete criteria that mark an image as saturated or non-informative and cause it to be discarded before consuming downlink budget.
4. Document the result as a concept-of-operations section, cross-referenced from the APSCO report rather than duplicated into it.

7.2 Data volume produced

7.2.1 Current status

- Capture resolution is established: 4056×3040 (approximately 12.3 MP) for full-resolution capture, 1920×1440 for preview.
- The mission downlink budget is known: S-band at 200 kbps, giving approximately 15 MB per 10-minute pass. The APSCO report already states explicitly that this is not compatible with continuous downlink of raw full-resolution images.
- No JPEG file-size estimate at the 4056×3040 capture resolution exists anywhere in the project.
- The number of images obtainable per operational period is an open item in the Camera System Report, pending both a file-size figure and an onboard storage capacity figure.

7.2.2 Objective

Quantify exactly how many images, at what resolution and compression level, can be produced, stored onboard, and downlinked per operational period.

7.2.3 Step-by-step tasks

1. Measure the actual JPEG file size at the 4056×3040 capture resolution with a simple bench test (capture one image, record the file size); no estimate currently exists for this.
2. Compute onboard storage capacity in number of storable images, using the project's assumed SD card size (64 GB or larger, per the installation guidance already in the repository).
3. Compute the downlink-limited image count per pass (approximately 15 MB per 10-minute pass divided by the measured JPEG file size).
4. Compare the storage-limited and downlink-limited image counts to identify which one is the actual bottleneck.
5. Feed the resulting number back into the capture-cadence decision in the Mode of Use section above, so the defined cadence stays realistic against what can actually be stored and downlinked.

Chapter 8

Stereo Camera Recalibration

This is not one of the six original tracking areas, but it blocks tasks inside Chapters 3 and 4, and is broken out on its own so that dependency is explicit rather than buried inside those chapters.

8.1 Current status

- The currently loaded calibration (`data/calibration/calibration_data.json`) is explicitly flagged as poor quality by its own metadata: reprojection error 1.53 px, `is_good_quality: false`, dated 2026-02-16, using 25 images.
- That calibration was computed at 1920×1440 , which predates the code fix that forces full-sensor 4056×3040 capture resolution. Even if its reprojection error were acceptable, it would not match the resolution the system now captures at.
- 23 additional timestamped calibration attempts exist under `data/calibration/`, spanning October 2025 to February 2026, indicating recalibration has been attempted repeatedly without yet producing a result flagged as good quality at the current capture resolution.
- The cameras are not currently physically installed in the dark enclosure, which is the immediate practical blocker to running a new calibration session at all.

8.2 Objective

Produce a calibration flagged as good quality by the existing validation logic, at the current 4056×3040 full-sensor capture resolution.

8.3 Step-by-step tasks

1. Physically mount the finalized camera hardware in the dark enclosure (shared step with Chapter 3).
2. Run a new calibration session (10 to 30 chessboard image pairs, 10×7 squares, 24 mm) at the current full 4056×3040 capture resolution.
3. Confirm the resulting calibration is flagged `is_good_quality: true` by the existing validation logic in `camera/camera_calibration.py`; if not, repeat with better lighting or more varied board angles/distances per the existing troubleshooting guidance.
4. Update `config/camera_config.py`'s auto-loaded calibration reference and confirm downstream code (stereo processing, point cloud generation) picks up the new calibration correctly.

Chapter 9

Dependency Map and Master Checklist

9.1 Why order matters more than dates here

Without fixed dates, the useful question is not "when" but "what can be worked on right now versus what is waiting on something else." Most chapters in this plan are independent of one another and can proceed in parallel today. Physical access to the camera hardware is the one recurring blocker that several chapters share.

9.2 Dependency diagram

Figure 9.1 separates tasks that can start immediately (top row, no shared blocker) from the single physical-access blocker (middle) and the tasks that wait on it (bottom row).

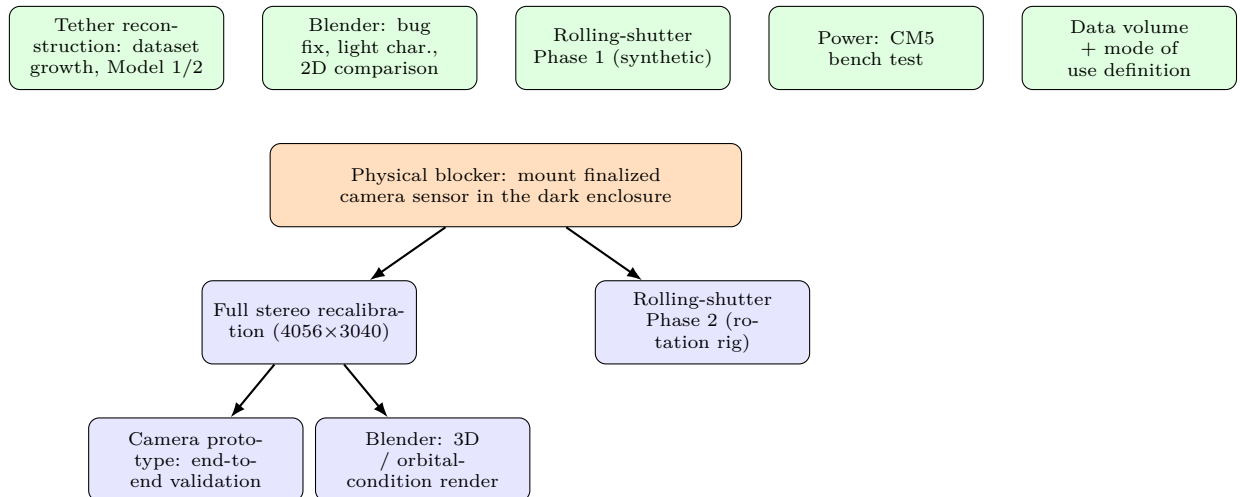


Figure 9.1: Dependency map. Green: can start immediately, no shared blocker. Orange: the one recurring physical blocker. Blue: waits on that blocker.

9.3 Master checklist

Table 9.1 lists every chapter's objective as a single line, so the whole plan's status can be read at a glance without opening every chapter.

Area	Objective	Blocked?
Tether reconstruction	Trained, validated Model 2; validated Model 1	No
Camera prototype	Sensor decision, physical build, validated	Final step only
Blender simulation	Lab-condition agreement confirmed; orbital render produced	Partially
Rolling-shutter correction	Correction validated synthetically and physically	Phase 2 only
Power consumption	Complete per-stage power budget	No
Mode of use	Defined trigger conditions and cadence per phase	No
Data volume	Quantified images per operational period	No
Recalibration	Good-quality calibration at full resolution	Yes, on camera mounting

Table 9.1: Master checklist across all tracking areas.

Chapter 10

Summary

Seven areas are tracked in this plan: tether reconstruction, new camera prototype, Blender simulation testing, rotation and rolling-shutter correction, power consumption, mode of use, and data volume produced. An eighth, stereo camera recalibration, is tracked separately because it blocks tasks inside two of the others.

Most of the work does not depend on physical camera access and can proceed now: growing the tether-reconstruction dataset, fixing the Blender tether-rendering bug and characterizing the dark-room light source, the rolling-shutter Phase 1 synthetic validation, CM5 power bench testing, and defining capture cadence and data-volume figures. The single recurring blocker is mounting a finalized camera sensor in the dark enclosure, which is required before recalibration, which in turn is required before the camera prototype can be fully validated and before the Blender scene can be compared in 3D.

This document has no fixed dates by design. As tasks are completed, this file should be updated directly (status bullets and checklist marked, step lists trimmed) rather than superseded by a new document, so it stays the single place to check current progress across all seven areas.